- *Application model*—The application model isn't really an ASP.NET Core model at all. It's typically a whole group of different services and classes and is more of a concept—anything needed to perform some sort of business action in your application. It may include the domain model (which represents the thing your app is trying to describe) and database models (which represent the data stored in a database), as well as any other, additional services.

  In the to-do list application, the application model would contain the complete list of to-do items, probably stored in a database, and would know how to find only those to-do items in the `open` category assigned to `andrew`.

- *Page model*—The `PageModel` of a Razor Page serves two main functions: it acts as the controller for the application by exposing page handler methods, and it acts as the view model for a Razor view. All the data required for the view to generate a response is exposed on the `PageModel`, such as the list of to-dos in the `open` category assigned to `andrew`.

  The `PageModel` base class that you derive your Razor Pages from contains various helper properties and methods. One of these, the `ModelState` property, contains the result of the model validation as a series of key-value pairs. You'll learn more about validation and the `ModelState` property in section 16.3.

These models make up the bulk of any Razor Pages application, handling the input, business logic, and output of each page handler. Imagine you have an e-commerce application that allows users to search for clothes by sending requests to the `/search/{query}` URL, where `{query}` holds their search term: